

04. Travel Agent - Tools

5. Implementazione degli Strumenti

5.1 Flights Finder Tool

Il tool per la ricerca voli è progettato per interfacciarsi con l'API di SerpApi (<https://serpapi.com/>) per ottenere informazioni sui voli.

Crea la cartella Tools e il file flights_finder.py

```
├─ README.md
├─ poetry.lock
├─ pyproject.toml
├─ tests
│   └─ __init__.py
└─ travel_agent_api
    ├── __init__.py
    ├── main.py
    ├── routes
    │   └─ chat_router.py
    └─ tools
        └─ flights_finder.py
```

Ecco l'implementazione dettagliata:

```
# flights_finder.py
import os
from dotenv import load_dotenv
from langchain_core.tools import tool
from serpapi import GoogleSearch
from pydantic import BaseModel, Field
from typing import Optional

load_dotenv()

class FlightsInput(BaseModel):
    departure_airport: str = Field(description="The departure airport code (IATA).")
    arrival_airport: str = Field(description="The arrival airport code (IATA).")
    outbound_date: str = Field(description="The outbound date (YYYY-MM-DD) e.g. 2024-12-13.")
    return_date: str = Field(description="The return date (YYYY-MM-DD) e.g. 2024-12-19.")
    adults: Optional[int] = Field(1, description="The number of adults. Defaults to 1.")
    children: Optional[int] = Field(0, description="The number of children. Defaults to 0.")

class FlightsInputSchema(BaseModel):
    params : FlightsInput

@tool(args_schema=FlightsInputSchema)
def flights_finder(params: FlightsInput):
    """
    This tool uses the SerpApi Google Flights API to retrieve flights info.

    Parameters:
    """
```

```
departure_airport (str): The departure airport code (IATA).
arrival_airport (str): The arrival airport code (IATA).
outbound_date (str): The outbound date (YYYY-MM-DD) e.g. 2024-12-13.
return_date (str): The return date (YYYY-MM-DD) e.g. 2024-12-19.
adults (int): The number of adults. Defaults to 1.
children (int): The number of children. Defaults to 0.
```

Returns:

dict: A dictionary containing the flights info. If the API call fails, it returns the error message as a string.

```
"""
```

```
try:
```

```
    params = {
        "api_key": os.getenv("SERPAPI_API_KEY"),
        "engine": "google_flights",
        "hl": "it",
        "gl": "it",
        "currency": "EUR",
        "stops": "1",
        "departure_id": params.departure_airport,
        "arrival_id": params.arrival_airport,
        "outbound_date": params.outbound_date,
        "return_date": params.return_date,
        "adults" : params.adults,
        "children": params.children
    }
```

```
    search = GoogleSearch(params)
```

```
    # Tracing
```

```
    print("*" * 80)
```

```
    print("flights_finder")
```

```
    print("*" * 80)
```

```
    return search.get_dict()
```

```
except Exception as e:
```

```
    return str(e)
```

Descrizione

Questo script Python implementa uno strumento per la ricerca dei voli utilizzando l'API di Google Flights attraverso SerpAPI. Lo strumento è strutturato per essere utilizzato come componente di LangChain, un framework per applicazioni basate su AI.

Il codice si articola in diverse parti fondamentali:

Il sistema inizia caricando le variabili d'ambiente necessarie, in particolare la chiave API di SerpAPI, utilizzando la libreria dotenv.

La struttura dei dati in ingresso è definita attraverso due classi Pydantic: FlightsInput e FlightsInputSchema. FlightsInput definisce tutti i parametri necessari per una ricerca voli:

- Codici IATA degli aeroporti di partenza e arrivo
- Date di andata e ritorno nel formato YYYY-MM-DD
- Numero di adulti (predefinito a 1)
- Numero di bambini (predefinito a 0)

La funzione principale flights_finder è decorata come tool di LangChain e accetta questi parametri strutturati. La funzione:

1. Prepara una richiesta all'API di Google Flights con parametri specifici come la lingua italiana (hl: "it"), la valuta in Euro, e un limite di 1 scalo
2. Esegue la ricerca attraverso SerpAPI
3. Restituisce i risultati in formato dizionario

Il sistema include anche una gestione degli errori che cattura e restituisce eventuali eccezioni come stringhe, garantendo che lo strumento non si blocchi in caso di problemi con l'API.

Questo tool può essere facilmente integrato in applicazioni più ampie basate su LangChain, permettendo di automatizzare le ricerche di voli all'interno di flussi di lavoro più complessi.

5.2 Hotels Finder Tool

Crea il file `hotels_finder.py` nella cartella `tools`

```
├─ README.md
├─ poetry.lock
├─ pyproject.toml
├─ tests
│   └─ __init__.py
└─ travel_agent_api
    ├── __init__.py
    ├── main.py
    ├── routes
    │   └─ chat_router.py
    └─ tools
        ├── flights_finder.py
        └─ hotels_finder.py
```

Il tool per la ricerca hotel implementa una funzionalità simile ma per le strutture ricettive:

```
# hotels_finder.py
import os
from langchain_core.tools import tool
from serpapi import GoogleSearch
from dotenv import load_dotenv
from pydantic import BaseModel, Field
from typing import Optional
from enum import IntEnum

load_dotenv()

class HotelClassEnum(IntEnum):
    TWO = 2
    THREE = 3
    FOUR = 4
    FIVE = 5

class HotelsInput(BaseModel):
    q: str = Field(description="Location of the hotel.")
    check_in_date: str = Field(description="The outbound date (YYYY-MM-DD) e.g. 2024-12-13.")
    check_out_date: str = Field(description="The return date (YYYY-MM-DD) e.g. 2024-12-19.")
    adults: Optional[int] = Field(1, description="The number of adults. Defaults to 1.")
    children: Optional[int] = Field(0, description="The number of children. Defaults to 0.")
    hotel_class: Optional[int] = Field(2, description="The hotel class available from 2 to 5 . Defaults to 2.")
```

```

class HoltesInputSchema(BaseModel):
    params : HotelsInput

@tool(args_schema=HoltesInputSchema)
def hotels_finder(params: HotelsInput):
    """
    This tool uses the SerpApi Google Hotels API to retrieve hotels info.

    Parameters:
        q (str): Location of the hotel.
        check_in_date (str): The outbound date (YYYY-MM-DD) e.g. 2024-12-13.
        check_out_date (str): The return date (YYYY-MM-DD) e.g. 2024-12-19.
        adults (int): The number of adults. Defaults to 1.
        children (int): The number of children. Defaults to 0.
        hotel_class (int): The hotel class available from 2 to 5. Defaults to 2.

    Returns:
        dict: A dictionary containing the hotels info. If the API call fails, it returns the
        error message as a string.
    """

    params = {
        "api_key": os.getenv("SERPAPI_API_KEY"),
        "engine": "google_hotels",
        "hl": "it",
        "gl": "it",
        "currency": "EUR",
        "q": params.q,
        "check_in_date": params.check_in_date,
        "check_out_date": params.check_out_date,
        "adults": params.adults,
        "children": params.children,
        "hotel_class": params.hotel_class,
        "num": 5
    }

    try:
        search = GoogleSearch(params)
        results = search.get_dict()["properties"]
    except Exception as e:
        print("Error:", e)
        results = str(e)

    # Tracing
    print("*" * 80)
    print("hotels_finder")
    print("*" * 80)

    return results

```

Descrizione

Questo script Python implementa uno strumento di ricerca hotel che sfrutta l'API Google Hotels attraverso SerpAPI. Il codice è strutturato per gestire in modo efficiente la ricerca di hotel basata su diversi parametri come località, date e numero di ospiti.

La struttura principale del codice si basa su tre componenti chiave:

1. Una classe enumerata `HotelClassEnum` che definisce le possibili classificazioni degli hotel (da 2 a 5 stelle).

2. Il modello di input `HotelsInput` che definisce e valida i parametri di ricerca:

- località dell'hotel
- data di check-in
- data di check-out
- numero di adulti (predefinito a 1)
- numero di bambini (predefinito a 0)
- classe dell'hotel (predefinita a 2 stelle)

3. La funzione principale `hotels_finder` che elabora la richiesta e interagisce con l'API.

Quando viene chiamata, la funzione `hotels_finder` prende i parametri forniti e costruisce una query appropriata per SerpAPI. La ricerca viene configurata per restituire risultati in italiano (parametri "hl" e "gl" impostati su "it") e i prezzi vengono mostrati in Euro. La funzione è impostata per restituire un massimo di 5 risultati per query.

Il codice include anche una gestione degli errori robusta: se qualcosa va storto durante la chiamata API, invece di interrompersi, cattura l'eccezione e restituisce il messaggio di errore come stringa. Questo rende lo strumento più affidabile e più facile da debuggare.

Per tracciare l'esecuzione, la funzione stampa dei separatori visivi (linee di asterischi) insieme al nome della funzione, facilitando il debug e il monitoraggio delle chiamate.

È importante notare che lo script richiede una chiave API valida per SerpAPI, che viene caricata dalle variabili d'ambiente attraverso il modulo `dotenv`. Questa è una best practice per la gestione sicura delle credenziali.

Lo strumento è stato progettato per essere utilizzato come parte di un sistema più ampio, come suggerito dall'uso del decoratore `@tool` e l'integrazione con `langchain_core`, rendendolo ideale per l'integrazione in applicazioni di automazione o chatbot.

5.3 Travel Plan Tool

Crea il file `chain_travel_plan.py` nella cartella `tools`

Questo tool genera piani di viaggio personalizzati utilizzando LLM:

```
# chain_travel_plan.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
from typing import Optional
from langchain_core.output_parsers import PydanticOutputParser

class TravelPlanInput(BaseModel):
    start_date: str = Field(description="The start date of the trip (YYYY-MM-DD) e.g. 2024-12-13.")
    end_date: str = Field(description="The end date of the trip (YYYY-MM-DD) e.g. 2024-12-19.")
    destination: str = Field(description="The destination of the trip.")
    adults: Optional[int] = Field(1, description="The number of adults. Defaults to 1.")
    children: Optional[int] = Field(0, description="The number of children. Defaults to 0.")
    travel_style: str = Field(description="The style of travel. e.g. adventure, relax, culture, backpacking, luxury, family-friendly.")
    budget: Optional[int] = Field(description="The total budget for the trip.")
    activities: str = Field(description="The preferred activities. e.g. culture, nature, food, shopping.")
    food_restriction: str = Field(description="Any food restrictions. e.g. vegetarian, gluten-free.")

class TravelPlanInputSchema(BaseModel):
```

```
params : TravelPlanInput
```

```
class TravelDayOutput(BaseModel):
    morning: str = Field(description="The activities for the morning.")
    afternoon: str = Field(description="The activities for the afternoon.")
    evening: str = Field(description="The activities for the evening.")

class TravelPlanOutput(BaseModel):
    travel_plan: list[TravelDayOutput]

@tool(args_schema=TravelPlanInputSchema)
def chain_travel_plan(params: TravelPlanInput) -> TravelPlanOutput:
    """
    Generates a comprehensive travel plan based on user input parameters.

    Parameters:
        params (TravelPlanInput): The input parameters for the travel plan
        including dates, destination, number of travelers, travel style, budget,
        preferred activities, and any food restrictions.

    Returns:
        TravelPlanOutput: The generated travel plan content.
    """

    model = ChatOpenAI(model_name="gpt-4o")

    system_prompt = f"""
    You are a travel expert.
    Your mission is to provide in-depth content on the topic to create a travel plan.
    The start date of the trip is {params.start_date}.
    The end date of the trip is {params.end_date}.
    The destination of the trip is {params.destination}.
    The number of adults is {params.adults}.
    The number of children is {params.children}.
    The style of travel is {params.travel_style}.
    The total budget for the trip is {params.budget}.
    The preferred activities are {params.activities}.
    Any food restrictions are {params.food_restriction}
    Use emojis to make your answers more engaging and friendly.
    Always strive to be approachable and helpful, offering the
    most accurate and useful information possible to users.
    """

    prompt = ChatPromptTemplate([("human", "{input}")])
    output_parser = PydanticOutputParser(pydantic_object=TravelPlanOutput)
    chain = prompt | model | output_parser
    result = chain.invoke(input=system_prompt)

    # Tracing
    print("*" * 80)
    print("chain_travel_plan")
    print("*" * 80)

    return result
```

Descrizione

Questo codice implementa un sistema di pianificazione viaggi utilizzando LangChain e OpenAI. Ecco come funziona:

Il sistema è strutturato attraverso tre componenti principali: la definizione degli input, l'elaborazione, e la formattazione dell'output.

Per quanto riguarda gli input, il sistema utilizza la classe `TravelPlanInput` che definisce tutti i parametri necessari per pianificare un viaggio:

- Date di inizio e fine nel formato YYYY-MM-DD
- Destinazione
- Numero di adulti e bambini
- Stile di viaggio (avventura, relax, cultura, ecc.)
- Budget
- Attività preferite
- Eventuali restrizioni alimentari

La funzione principale `chain_travel_plan` prende questi parametri e li elabora attraverso una catena di operazioni:

1. Crea un'istanza del modello ChatGPT (GPT-4)
2. Genera un prompt di sistema che include tutti i parametri del viaggio
3. Configura una catena di elaborazione che:
 - Prende il prompt come input
 - Lo invia al modello linguistico
 - Elabora la risposta attraverso un parser

L'output viene strutturato attraverso la classe `TravelDayOutput` che divide ogni giornata in tre parti:

- Attività mattutine
- Attività pomeridiane
- Attività serali

Il sistema utilizza emoji per rendere il piano più coinvolgente e amichevole.

Un aspetto interessante è l'uso di Pydantic per la validazione dei dati: questo assicura che tutti i parametri inseriti rispettino i formati richiesti e che i valori opzionali abbiano dei default sensati (ad esempio, 1 adulto e 0 bambini come valori predefiniti).

La catena di elaborazione (chain) rappresenta un flusso di lavoro lineare dove:

4. I dati dell'utente vengono validati
5. Vengono inseriti in un prompt strutturato
6. Il prompt viene elaborato dal modello AI
7. La risposta viene parsata in un formato strutturato per giornata

Questo approccio modulare rende il codice facilmente manutenibile e estensibile, permettendo di modificare singoli componenti (come il modello AI o il formato dell'output) senza dover riscrivere l'intera logica.

5.4 Historical Expert Tool

Crea il file `chain_historical_expert.py` nella cartella tools

Questo tool utilizza un modello di intelligenza artificiale per fornire contenuti approfonditi su un argomento storico specifico.

```
# chain_historical_expert.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langchain.prompts import ChatPromptTemplate

@tool
```

```
def chain_historical_expert(input_text: str) -> str:
    """
    Questo tool utilizza un modello di intelligenza artificiale per fornire contenuti
    approfonditi su un argomento storico specifico.

    Args:
        input_text (str): Il testo dell'argomento per il quale si vuole ottenere il
        contenuto.

    Returns:
        str: Il contenuto generato dal modello.
    """

    model = ChatOpenAI(model_name="gpt-4o")

    system_prompt = """
    You are an historical expert.
    Your mission is to provide in-depth content on the topic,
    answer questions, and act as an assistant.
    Use emojis to make your answers more engaging and friendly.
    Always strive to be approachable and helpful, offering the
    most accurate and useful information possible to users.
    """

    prompt = ChatPromptTemplate([
        ("system" , "{system_prompt}"),
        ("user" , "{input}")
    ])

    chain = prompt | model
    result = chain.invoke({
        "input":input_text,
        "system_prompt":system_prompt
    })

    # Tracing
    print("*" * 80)
    print("chain_historical_expert")
    print("*" * 80)

    return result
```

Descrizione

Questo codice implementa uno strumento specializzato che funge da esperto storico utilizzando l'intelligenza artificiale. Ecco come funziona:

La funzione principale `chain_historical_expert` è decorata con `@tool`, che la rende uno strumento utilizzabile all'interno dell'ecosistema LangChain. Accetta un parametro di input testuale e restituisce una risposta elaborata.

Il processo si svolge in diversi passaggi:

1. Inizialmente, viene creata un'istanza del modello ChatOpenAI, configurata per utilizzare GPT-4.
2. Viene definito un "system prompt" che stabilisce il comportamento base del modello. In questo caso, viene istruito a comportarsi come un esperto di storia, utilizzare emoji per rendere le risposte più coinvolgenti e mantenere un tono amichevole e professionale.
3. Viene creato un template per il prompt utilizzando ChatPromptTemplate. Questo template combina due elementi:
 - Il prompt di sistema che definisce il comportamento

- L'input dell'utente che contiene la domanda o l'argomento specifico
4. Viene costruita una "catena" di elaborazione usando l'operatore pipe (|) che collega il prompt al modello.
 5. La catena viene eseguita attraverso il metodo `invoke()`, passando sia l'input dell'utente che il prompt di sistema.
 6. Per scopi di debugging e monitoraggio, viene stampata una linea di separazione e il nome della funzione.
 7. Infine, viene restituita la risposta generata dal modello.

In pratica, quando si utilizza questo strumento, fornendo un testo relativo a un argomento storico, si ottiene una risposta dettagliata e ben strutturata, arricchita da emoji, che mantiene un tono amichevole pur essendo professionale e accurata.